

Milestone 3 Report

Smart MCQ Solver Challenge

Course: BSDA2001P – Introduction to Deep Learning and Generative AI

Name: Soumyadip Hazari

Student ID: 24f3004946

Objective

This milestone focuses on the core mechanics of Retrieval-Augmented Generation (RAG). Where we will explore how to query a vector index for relevant context and use a Cross-Encoder ([cross-encoder/ms-marco-MiniLM-L-6-v2](#)) to re-rank the results for maximum accuracy. Next we will run side-by-side A/B tests to compare a model's zero-shot inference with and without this context. Finally, we will then explore the physical limits of context windows (tuning the correct number of chunks to retrieve) and demonstrate the catastrophic dangers of feeding an LLM incorrect data.

Overall this introduces Retrieval-Augmented Generation (RAG) why it is needed and how it improves over the transformer-only approach from Milestone 2

Work Performed

1. Import Required Libraries and Loading Dataset –

The first step of the milestone contains the importing necessary libraries required for the data processing, vector search, transformer models and evaluation for which pandas, numpy, FAISS, SentenceTransformers, Transformers and Scikit-learn were imported.

Later By using pandas we loaded train.csv, test.csv and sample_submission.csv.

2. Knowledge Base Construction –

We extracted only the correct answer text from each training sample rather than storing the complete training rows.

For every row:

- Read the correct answer label(A-E).
- Retrieve the corresponding option text.
- Store the text inside a Python list called Knowledge Base (KB).

After processing all rows

- Total documents - 2000

```
print("Creating Knowledge Base...")

kb = []

for _, row in train.iterrows():

    correct_letter = row["answer"]          # A, B, C, D or E

    correct_option = str(row[correct_letter])

    kb.append(correct_option)

print("Knowledge Base successfully created.")
print("Total documents:", len(kb))
```

3. Generating Dense Embeddings –

For the embeddings we used **all-MiniLM-L6-v2** (This is a sentence-transformers model which maps sentences & paragraphs to 384 dimensional dense vector space and can be used for tasks like clustering or semantic search.)

```
model = SentenceTransformer("all-MiniLM-L6-v2")

then ,
kb_embeddings = model.encode(kb, show_progress_bar=True)
print("Embeddings created.")
print("Embedding shape:", kb_embeddings.shape)
```

Which then converts knowledge base into a dense semantic embedding and the resulting dimension of the embedding matrix is (2000,384)

4. Building FAISS Vector Index

The embeddings that were generated were then inserted into **FAISS IndexFlatL2** vector database (IndexFlatL2 is a simplest brute-force index in FAISS library)(Where FAISS is a library which allows users to quickly search for embeddings of documents that are similar to each other).

```
index = faiss.IndexFlatL2(kb_embeddings.shape[1])
index.add(kb_embeddings)
print("Number of indexed documents: ", index.ntotal)
```

Which then gives 2000 indexed documents.

5. Baseline Zero-Shot Classification

The huggingface model facebook/bart-large-mnli were loaded using the zero-shot classification pipeline.

```
zs = pipeline("zero-shot-classification", model =  
"facebook/bart-large-mnli")  
print("Zero-shot classifier loaded")
```

For Question 1,
only the prompt was supplied together with the five answer choices.
The model ranked the options according to their probability scores.

```
result = zs(prompt_150, candidate_labels = labels_150)  
ground_truth_score = result["scores"][  
    result["labels"].index(ans_150)  
]  
print("Ground-truth probability: ", round(ground_truth_score, 3))
```

6. Semantic Retrieval using FAISS

The prompt from row 150 was converted into an embedding using the same MiniLM model.

Then this query embedding was searched against the FAISS index. And we retrieved the top-10 most similar documents and the true document appeared only at Rank 10 which demonstrates that dense retrieval alone is not always sufficient.

```
query_embedding = model.encode([prompt_150])  
k = 10  
distances, indices = index.search(query_embedding, k)  
true_doc = 150  
rank = list(indices[0]).index(true_doc) + 1  
print("Rank of the document: ", rank)
```

7. Cross-Encoder Re-ranking

The retrieved documents were passed to the Cross-Encoder **cross-encoder/ms-marco-MiniLM-L-6-v2**

Then each document was paired with the question. Unlike the Bi-Encoder, the Cross-Encoder jointly processed both the question and the document, allowing the attention mechanism to evaluate semantic relationships directly. After reranking, the correct document moved from Rank 10 to Rank 1

This clearly demonstrated the effectiveness of reranking.

```
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
```

```

retrieved_indices = indices[0]

docs_10 = [kb[i] for i in retrieved_indices]

pairs = [[prompt_150, doc] for doc in docs_10]

ce_scores = cross_encoder.predict(pairs)

ranking = sorted(zip(retrieved_indices, ce_scores), key=lambda x:
x[1], reverse = True)

for rank, (doc_id, score) in enumerate(ranking, start=1):

    print(

        f"Rank {rank}: "

        f"Document {doc_id}, "

        f"Score = {score:.4f}"

    )

```

8. Context Window Analysis

To understand context length, five retrieved documents were concatenated together.

The final prompt was created as

Context:

[Retrieved Documents]

Question:

[Prompt]

The combined text was tokenized using bert-base-uncased. The total number of generated tokens was 216

This experiment demonstrated how retrieved documents increase the context window of transformer models.

```

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

tokens = tokenizer(final_text, truncation = False)

total_tokens = len(tokens["input_ids"])

print("Total Tokens: ", total_tokens)

```

9. Retrieval-Augmented Generation (RAG)

The correctly retrieved document was injected into the prompt as external context. Where the final prompt become

Context:

Correct Document

Question:

Prompt

Now the same zero-shot classifier was executed again. And the probability assigned to the correct answer increased from 0.384 to 0.986. Now showed that supplying relevant external knowledge significantly increased model confidence.

```
rag_prompt = (f"Context: {true_document}", f"Question:{prompt_150}")
rag_result = zs(rag_prompt, candidate_labels = labels_150)
ground_truth_score = rag_result["scores"][
    rag_result["labels"].index(ans_150)
]
print("Ground-truth probablity:", round(ground_truth_score, 3))
```

10. Adversarial Retrieval

To study the limitations of RAG, an unrelated document from KB[999] was deliberately injected as context.

Then the zero-shot classifier was executed again.

The confidence of the correct answer changed to 0.528

The observation reinforces the principle: **Garbage In → Garbage Out**

```
wrong_document = kb[999]

adversarial_prompt = (f"Context: {wrong_document}"
    f"Question: {prompt_150}")

adversarial_result = zs(adversarial_prompt, candidate_labels =
    labels_150)

ground_truth_score = adversarial_result["scores"][
    adversarial_result["labels"].index(ans_150)
]

print("Groung-truth probability: ", round(ground_truth_score, 3))
```

11. Retrieval Evaluation (Hit Rate)

For first we need to know Hit Rate –

1. First we need to get the first 100 rows of train.csv
2. then taking the prompts we need to embed using all-MiniLm-L6-v2
3. after retrieving the top 5 documents from FAISS we need to check whether the exact correct answer text for the row appears among those retrieved documents
4. if yes --> Hit
5. otherwise didn't count

With that the Hit Rate was calculated as 73.0%, This metric measured the effectiveness of the retrieval stage independently of the language model.

```
hits = 0

k = 5

for i in range(100):

    row = train.iloc[i]

    prompt = str(row["prompt"])

    correct_answer = str(row[row["answer"]])

    query_embedding = model.encode([prompt])

    distances, indices = index.search(query_embedding, k)

    retrieved_docs = [kb[idx] for idx in indices[0]]

    if any(correct_answer in doc for doc in retrieved_docs):

        hits +=1

hit_rate = (hits/100)*100

print("Hits: ", hits)

print("Hit Rate: ", round(hit_rate, 1), "%")
```

12. Complete RAG Pipeline Evaluation

At the end a complete Retrieval-Augmented Generation pipeline was constructed

1. Embed the prompt.
2. Retrieve Top-5 documents using FAISS.
3. Re-rank documents using the Cross-Encoder.
4. Select the highest-ranked document.
5. Construct the augmented prompt.
6. Execute zero-shot classification.
7. Rank the answer choices.
8. Compute MAP@3.

The average MAP@3 over the first twenty training samples is

Key Results

Question	Task Performed	Result
Q1	Zero-shot classification without context	0.384
Q2	FAISS retrieval rank of the correct document	10
Q3	Cross-Encoder reranked position	1
Q4	Total BERT tokens after context augmentation	216
Q5	Zero-shot classification with correct context	0.986
Q6	Zero-shot classification with adversarial context	0.528
Q7	Retrieval Hit Rate@5 (First 100 samples)	73.0%
Q8	Average MAP@3 (First 20 samples)	0.975